

TypeScript

Brainster Web Development Academy



Table of contents

- 01 Intro
- 02 Project Setup
- 03 Core Types
- 04 Union/Literal Types
- 05 Function types
- 06 Generics
- 07 Wrap up



What is TypeScript

- Programming language built up on JavaScript
- Adds new features and advantages to JavaScript
- In the end Typescript is compiled to JavaScript
- In one word it is a typed superset of JavaScript
- Everything that we learned about JS syntax stays in TS

3



Why TypeScript

- Remove unexpected behavior at runtime
- Less logical bugs
- Helps developers write better code
- JavaScript which scales

Disadvantages

- Browsers and Nodejs can't execute TypeScript
- Transpiler is need to translate TS to JS
- Additional code needs to be written

JavaScript

```
function add(a, b) {  
    return a + b  
}  
  
console.log(add('5', 10))
```

TypeScript

```
function add(a: number, b: number) {  
    return a + b  
}  
  
console.log(add('5', 10))  
// Argument of type 'string' is not assignable to  
parameter of type 'number'
```



TypeScript Features

- Adds types to variables!
- Detects error in code without running it (static type checker)
- Better code completion
- Modern JavaScript features compiled for older browsers
- Contains non JavaScript features like Generics or Interfaces
- Rich Configuration Options



Exercise I

- Create two inputs in HTML and a button with text Add
- In **JavaScript** create a function that will accept two parameters and simply returns the sum of those two parameters
- Select the inputs and the button in javascript
- Add event listener that will execute the add function on button click which as parameters will take the values from the inputs
- Add the result of the function into the document



TypeScript Setup

- Nodejs
- Documentation ([typescript](#))
- Globally Installing Typescript
npm install -g typescript
- “tsc” command can be used everywhere
tsc [filename].ts
- Remember browsers can't execute .ts files, always Include .js files into your html
- Visual studio code extension - optional



Exercise II

- Update exercise I to use **TypeScript** in order to prevent the unexpected behavior and additional workarounds



Core Types

- Type inference
- Number - all numbers
- String - all text values
- Boolean - just two values true or false
- Objects - key value pairs
- Array - collection of items
- Tuple - exactly two items in array
- Enums - Global constants of numbers with label
- Any - Store anything



Core Types - Objects

- Beside regular object definition, sometimes we don't know the properties ahead of time, in this case we can use `Record<Keys, Type>`

TypeScript

```
const myObj: Record<string, number> = {  
  'myProperty1': 11,  
  'myProperty2': "SomeValue", // Type 'string' is not assignable to type 'number'  
}
```





Core Types - Objects

Sometimes when we define objects we want to make some properties optional, we can achieve that using “?” after property name:

TypeScript

```
const myObj: {  
  name: string;  
  age?: number;  
  address: {  
    street: string;  
    number?: number;  
    unit?: string;  
  };  
} = {  
  name: "MyName",  
  address: {  
    street: "MyStreet",  
  },  
};
```

Or using Partial we can make every property to be optional

TypeScript

```
type UserType = {  
  name: string;  
  age: number;  
  address: {  
    street: string;  
    number: number;  
    unit: string;  
  };  
};  
  
const myObj: Partial<UserType> = { name: "MyName" };
```

Union Types

TypeScript

```
let variable: string | number

variable = 'Some string value'
variable = 1001
variable = true // Type 'boolean' is not assignable to type 'string | number'
```

- In this code, “variable” accepts only strings or numbers
- It can be extended to accept more types using the | sign
- Useful when we have to specify different behaviour for different type of variables



Literal Types

TypeScript

```
let variable: 'as text' | 'as number' | 0

variable = "as text"
variable = "as number"
variable = 0
variable = 'some other string'
// Type '"some other string"' is not assignable to type '0 | "as text" | "as number"'
```

- In this code, “variable” accepts only the exact strings or the exact number
- It can be extended to accept more types using the | sign
- Useful because we as developers don't have to remember the exact phrases, we get autocompletion from TypeScript



Type Aliases

TypeScript

```
type VariableTypes = 'as text' | 'as number' | 0
type InputTypes = number | boolean
type InputTypesExtended = InputTypes | VariableTypes | 'none'

let variable: InputTypesExtended

variable = "as text"
variable = 100
variable = 'none'
variable = true
variable = 'some other string' // error
// Type '"some other string"' is not assignable to type 'InputTypesExtended'
```

- In this code, “variable” accepts only what is covered with inputTypes Extended



Exercise III

- Using TypeScript, fetch all users from “<https://jsonplaceholder.typicode.com/users>” and render them in table
- You can create the table in html but the data should be created dynamically in typescript
- Use id, name, username, email, company name as table columns



Homework

- Extend on previous exercise
- When clicking on each column header the table rows should be sorted toggling ascending and descending

